ML Track Plan:

Isolation Forest Detection and Evaluation (S4 and S5)

Team:	Rushikesh, Sayali, Pooja (All 3 on the ML track)
Document Status:	Final Redesigned Implementation Plan
Target File:	`ML_implementation.md`

1. Executive Summary & Core Objective

The ML track implements an **Isolation Forest anomaly detector** operating alongside our existing deterministic threshold detector within the continuous monitoring loop (`backend/app/services/detection/scheduler.py`).

Key Design Principles:

1. **ML-Primary Incident Creation:** Incidents are created directly and primarily by the **Isolation Forest Anomaly Score**. When the composite anomaly score exceeds the calibrated decision boundary for *N* consecutive evaluation cycles, an incident is triggered.
2. **Single Source of Truth:** `backend/app/ml/features.py` defines all feature transformations and Prometheus query strings used across training, validation, and real-time inference to eliminate feature drift.
3. **Thresholds as Resilience Fallback Only:** Static rule thresholds do not compete with or duplicate ML alerts. If `isolation_forest.pkl` is missing or corrupt, the platform logs a clean warning and falls back to static threshold detection with zero downtime.
4. **Calibrated Thresholds:** No arbitrary `0.95` magic thresholds. The score threshold is empirically calibrated on validation data (`N2` run) at the 99.5th percentile + 0.02.
5. **Lean Runtime Dependencies:** Only `scikit-learn`, `numpy`, and `joblib` are loaded at backend runtime. `pandas` and data visualization tools are kept exclusively in the offline training/evaluation tooling (`ml/requirements-train.txt`).

2. Review Findings & Audit Corrections

#	Subsystem	Initial Inaccuracy / Issue	Redesigned Plan Fix
1	**CPU Metric**	Initial draft suggested `simulated_cpu_usage` (a manual gauge).	Uses real container process CPU: `rate(process_cpu_seconds_total{job="demo-app"}[1m]) * 100` (percent of one core).
2	**Restart Counter**	`container.restart` only fires on explicit manual restart commands, missing crashes.	Counts actual Docker crash events: `container.die` + `container.oom` from the PostgreSQL `events` table over 15m.

#	Subsystem	Initial Inaccuracy / Issue	Redesigned Plan Fix
3	**Anomaly Score Threshold**	`0.95` was assumed as threshold. In `scikit-learn`, `score_samples()` returns negative values. Inverting it gives values where anomalies rarely hit `1.0`.	Empirically calibrate score threshold on `normal_validate.csv` at 99.5th percentile + 0.02.
4	**Code Drift**	Initial thought kept separate scripts in `ml/` and `backend/`.	Unified feature logic in `backend/app/ml/features.py` shared between offline data extraction and online scheduler.
5	**Runtime Bloat**	Heavy data analysis libraries (e.g., `pandas`, `matplotlib`) risking memory limit breaches.	Runtime backend container pins only `numpy`, `scikit-learn`, `joblib`. `pandas` is restricted to `ml/requirements-train.txt`.
6	**Data Scarcity**	A 30-minute run at 15s yields only ~120 rows, insufficient for isolating multidimensional normality.	**60-minute training run (`N1`)** with stepped traffic (3, 5, 8 req/sec) + **20-minute validation run (`N2`)**.

3. The 8 Telemetry Features

Every 15-second cycle over a **5-minute rolling window** (W = 5m, requiring ≥ 12 valid data points):

#	Feature Key	Mathematical Definition	Data Source
1	`cpu_pct_mean_5m`	Rolling mean of `demo-app` process CPU %	Prometheus (`process_cpu_seconds_total`)
2	`cpu_pct_std_5m`	Standard deviation of `demo-app` process CPU %	Prometheus (`process_cpu_seconds_total`)
3	`latency_p95_mean_5m`	Rolling mean of P95 response latency (seconds)	Prometheus (`http_request_duration_seconds`)
4	`latency_p95_std_5m`	Standard deviation of P95 response latency	Prometheus (`http_request_duration_seconds`)
5	`error_rate_5m`	Percentage of 5xx HTTP responses over 5m	Prometheus (`http_requests_total`)
6	`request_rate_mean_5m`	Average requests per second over 5m	Prometheus (`http_requests_total`)
7	`container_die_count_15m`	Count of `container.die` + `container.oom` events (15m window)	PostgreSQL (`events` table)
8	`data_dir_growth_5m`	Net change (delta in bytes) in `demo_data_dir_bytes` over 5m	Prometheus (`demo_data_dir_bytes`)

4. Multi-Developer Work Allocation (Rushikesh, Sayali, Pooja)

Order of Work & Responsibility Matrix

Round	Rushikesh (Lead/Infra)	Sayali (Data/Model)	Pooja (Eval/UI)
R0 (Day 1)	**M0:** Shared Contract **M2:** Feature Maths (`backend/app/ml/`)	**M0:** Shared Contract **M1:** 60m Baseline Data Run (`N1`,`N2`)	**M0:** Shared Contract **M4:** Eval Harness (`ml/evaluation/`)
R1 (Day 1)	**M5:** Detector Class (`AnomalyDetector`)	**M3:** Baseline Exporter to CSV	**M7:** Incident UI (Anomaly score badge)
R2 (Day 2)	**M8:** Scheduler Integration (Hybrid)	**M6:** Train & Calibrate Model	**M7:** Wire UI to live backend incident data
R3 (Day 2)	Unit Tests & Fixes for M5 & M8	**M9:** Dry Runs & **Model Freeze**	Live verification of M4 with test scenario
R4 (Day 3)	**M11:** Clean Clone Verification	Dual-rater for AI RCA evaluation	**M10:** Run 5 Faults x3 & Generate `results.md`
R5 (Day 4)	Code Review & Final Quality Assurance	Model Card audit & Boundary verification	**M12:** Demo Script (`DEMO_SCRIPT.md`)

5. Detailed Task Cards

M0. Preflight & Shared Contract

- **Assignee:** Rushikesh, Sayali, Pooja (Joint 45-minute task)
- **Branch:** `feature/ml-contract`
- **File Created:** `docs/ML_CONTRACT.md`
- **Steps:**
  1. Audit current system parameters: verify `DETECTION_INTERVAL_SECONDS` (15), `DETECTION_CONSECUTIVE_BREACHES` (3), `DETECTION_RECOVERY_CHECKS` (3).
  2. Document the exact names, formulas, and ordering of the 8 features.
  3. Define detection modes: `threshold`, `ml`, and `hybrid`.
  4. Specify the schema for `ml/traces/detection_trace.jsonl` and database fields on `Incident`.
- **Done When:** `docs/ML_CONTRACT.md` is reviewed and merged into `dev`.

M1. Baseline Normal Data Run

- **Assignee:** Sayali
- **Branch:** `feature/ml-runs`
- **File Created:** `ml/evaluation/run_log.csv`
- **Steps:**
  1. Configure machine to prevent sleep/hibernation.
  2. Start clean stack: `docker compose -f docker-compose.yml -f docker-compose.demo.yml up -d`.
  3. Execute **N1 (Training Run - 60 mins)**: Step traffic generator rate at 20m intervals: `3`, `5`, and `8` req/sec.

4. Execute **N2 (Validation Run - 20 mins)**: Set baseline rate at `5` req/sec with short fluctuations to `3` and `8` req/sec.
5. Log UTC timestamps (`start_time`, `end_time`) and parameters to `run_log.csv`.

- **Done When:** Prometheus and Postgres contain clean, continuous data for both windows with zero container restarts.

M2. Feature Engineering & Query Library

- **Assignee:** Rushikesh
- **Branch:** `feature/ml-features`
- **Files:** `backend/app/ml/__init__.py`, `backend/app/ml/features.py`, `backend/tests/test_ml_features.py`
- **Steps:**
  1. Write pure mathematical functions (using `numpy` only) to compute rolling mean, standard deviation, error rates, and delta.
  2. Enforce minimum point requirement: return `None` + explicit reason if a window has < 12 points.
  3. Implement unified query generator functions for Prometheus queries and PostgreSQL event lookups.
  4. Add unit tests with synthetic fixture vectors (normal window, missing points, counter resets).
- **Done When:** `pytest backend/tests/test_ml_features.py` passes with 100% test success.

M3. Baseline Exporter

- **Assignee:** Sayali
- **Branch:** `feature/ml-collector`
- **Files:** `ml/collect_baseline.py`, `ml/requirements-train.txt`, `ml/data/normal_train.csv`, `ml/data/normal_validate.csv`
- **Steps:**
  1. Build CLI script accepting `--start`, `--end` (UTC), and `--out`.
  2. Query Prometheus range API (15s step) and Postgres `events` using functions from `backend/app/ml/features.py`.
  3. Export rows to CSV containing the 8 features and raw values. Automatically drop incomplete rows and log the count.
- **Done When:** Generates `normal_train.csv` (~240 rows) and `normal_validate.csv` (~80 rows).

M4. Automated Evaluation Harness

- **Assignee:** Pooja
- **Branch:** `feature/ml-harness`
- **Files:** `ml/evaluation/run_scenarios.py`, `ml/evaluation/report.py`, `ml/evaluation/rubric.md`
- **Steps:**
  1. Write runner script to automate fault execution:
     - 3-minute warm-up.
     - Execute target scenario script (`infrastructure/scripts/*.ps1`).
     - Poll until incident opens and auto-resolves (timeout: 10 mins).
     - Trigger AI RCA analysis endpoint.
     - 3-minute cooldown.

2. Append structured metrics per run to `ml/evaluation/runs.csv`.
3. Create `report.py` to compile statistical markdown tables and comparative plots.

- **Done When:** Dry-run test of `run_scenarios.py` successfully executes `cpu-stress.ps1` and records full metrics.

M5. AnomalyDetector Inference Engine

- **Assignee:** Rushikesh
- **Branch:** `feature/ml-detector`
- **Files:** `backend/app/ml/inference.py`, `backend/requirements.txt`, `docker-compose.yml`, `backend/app/core/config.py`, `backend/tests/test_ml_inference.py`
- **Steps:**
  1. Add exact pinned versions of `scikit-learn`, `numpy`, and `joblib` to `backend/requirements.txt`.
  2. Implement `AnomalyDetector`:
     - `load(model_dir)`: Loads `isolation_forest.pkl` and validates schema against `feature_metadata.json` (checks SHA-256 and library versions).
     - `score(features)`: Computes score $s = -\text{score_samples}(X)$, checks against threshold, and identifies up to 3 `top_features` by z-score deviation.
  3. Ensure resilience: Catch all exceptions internally and return `is_anomaly=False`, `score=None` with diagnostic logging.
  4. Mount `./ml/models:/models:ro` into the backend service in `docker-compose.yml`.
- **Done When:** Comprehensive unit tests pass covering corrupt models, missing files, dimension mismatches, and normal scoring.

M6. Model Training & Calibration

- **Assignee:** Sayali
- **Branch:** `feature/ml-train`
- **Files:** `ml/train.py`, `ml/calibrate.py`, `ml/models/isolation_forest.pkl`, `ml/models/feature_metadata.json`, `ml/MODEL_CARD.md`, `ml/tests/test_train.py`
- **Steps:**
  1. `train.py`: Fit `IsolationForest(n_estimators=200, max_samples='auto', contamination='auto', random_state=42)` on `normal_train.csv`.
  2. `calibrate.py`: Score `normal_validate.csv`. Set calibrated threshold at:

     $$Threshold = P_{99.5}(scores) + 0.02$$

  3. Write `feature_metadata.json` containing feature names, training means, standard deviations, calibrated threshold, and SHA-256 hash.
  4. Commit model artifacts and publish `MODEL_CARD.md`.
- **Done When:** Reproducibility tests confirm exact score consistency on golden test vectors.

M7. Frontend Incident Experience

- **Assignee:** Pooja
- **Branch:** `feature/ml-ui`

- **Files:** `frontend/src/pages/IncidentsList.tsx`, `frontend/src/pages/IncidentDetailView.tsx`, `frontend/src/types/index.ts`
- **Steps:**
  1. Show primary badge on incident cards: `Flagged by: ML Anomaly Detector` (or `Fallback: Threshold` if model was offline).
  2. Render Anomaly Score Radial Gauge / Progress Bar on the incident detail view indicating distance from the decision boundary.
  3. Display "Top Contributing Signals" breakdown card showing the z-score deviations of anomalous features.
  4. Display the model version and calibrated threshold boundary.
- **Done When:** `npm run build` succeeds and UI updates dynamically when an ML-flagged incident is selected.

M8. Detection Scheduler Integration

- **Assignee:** Rushikesh
- **Branch:** `feature/ml-scheduler`
- **Files:** `backend/app/services/detection/scheduler.py`, `backend/app/core/config.py`, `docker-compose.yml`, `backend/tests/test_scheduler.py`
- **Steps:**
  1. Instantiate `AnomalyDetector` during scheduler boot.
  2. Add `DETECTION_MODE` environment configuration: `ml | threshold | hybrid` (Default: `ml`).
     - In `ml` **mode** (Primary): An incident is created when the **ML Anomaly Score** breaches the calibrated threshold for *N* consecutive evaluation cycles.
     - In `threshold` **mode** (Fallback): Used automatically if the model file is missing or fails to load.
  3. In each 15s evaluation loop:
     - Query 5-minute metric windows and compute feature vector using `features.py`.
     - Execute ML scoring.
     - Append line to rotating `ml/traces/detection_trace.jsonl`.
  4. When an anomaly is detected, create an incident with `metric_name='anomaly_score'`, `metric_value=anomaly_score`, `threshold=calibrated_threshold`, and `affected_metrics=top_features`.
  5. Populate `anomaly_score`, `affected_metrics`, `model_version`, and `detection_source='ml'` on created `Incident` records.
- **Done When:** A fault scenario (e.g., CPU stress) causes the ML score to breach the calibrated threshold and creates an incident flagged by `detection_source='ml'`.

M9. Dry Runs & Model Freeze

- **Assignee:** Sayali (Assisted by Rushikesh)
- **Branch:** `feature/ml-freeze`
- **Files:** `ml/evaluation/dry_runs.md`
- **Steps:**
  1. Execute each of the 5 fault scripts once against the integrated platform.
  2. Verify that feature vectors react as expected and anomaly scores rise above threshold during faults.
  3. Fine-tune hyperparameter boundaries if any fault is missed.

4. Tag git commit `ml-freeze`. Lock the model artifact; only critical P0 bug fixes permitted thereafter.

- **Done When:** `dry_runs.md` records 5 successful test runs and git tag `ml-freeze` is created.

M10. Comprehensive Evaluation Execution (S5)

- **Assignee:** Pooja (Sayali as independent rater)
- **Branch:** `feature/ml-results`
- **Files:** `ml/evaluation/runs.csv`, `ml/evaluation/results.md`, `ml/evaluation/plots/`
- **Steps:**
  1. Execute **N3 (30-minute normal run)**: Measure and document false alarm rate per hour.
  2. Run the 5 fault scenarios **3 times each** (15 total runs) using `run_scenarios.py`:
     - CPU Stress
     - Container Crash
     - Database Stop
     - Latency Spike
     - Disk Fill
  3. Evaluate AI RCA accuracy using the dual-rater scoring rubric in `rubric.md`.
  4. Compile final comparative matrices into `ml/evaluation/results.md` (Time-to-Detect: Threshold vs. ML).
- **Done When:** `results.md` contains 15 verified scenario runs + normal run statistics without manual estimates.

M11. Clean Clone & Operational Readiness

- **Assignee:** Rushikesh
- **Branch:** `feature/ml-release-check`
- **Files:** `README.md`, `.gitignore`, `docs/ML_CONTRACT.md`
- **Steps:**
  1. Clone repository to a temporary directory and execute clean bootstrap:

     ```
     docker compose -f docker-compose.yml -f docker-compose.demo.yml up -d --build
     ```

  2. Validate model auto-loading from `./ml/models/`.
  3. Simulate missing model file: Confirm backend logs clean warning and falls back to threshold detection without 500 errors.
  4. Add 5-line ML operation guide to `README.md`.
- **Done When:** Clean clone test passes completely on a freshly launched stack.

M12. Final Results Review & Demo Script

- **Assignee:** Pooja (Reviewed by Sayali & Rushikesh)
- **Branch:** `feature/ml-demo`
- **Files:** `docs/DEMO_SCRIPT.md`, `ml/evaluation/results.md`
- **Steps:**
  1. Add live ML demonstration sequence to `docs/DEMO_SCRIPT.md`:
     - Show normal anomaly score baseline (≈ 0.15 - 0.30).

- Inject fault and show real-time score rise above threshold (> 0.65).
- Inspect incident detail showing ML attribution badge and top contributing signals.
- Demonstrate fallback by toggling `DETECTION_MODE=threshold`.

2. Document experimental limitations in `results.md`.

- **Done When:** Entire demo sequence can be reproduced by an independent operator end-to-end.

6. Execution Rules & Quality Gates

1. **One Task = One Branch = One PR:** Never commit directly to `dev` or merge your own pull request.
2. **Deterministic Data Only:** Never use synthetic data for normal baseline training; synthetic data is strictly for filling unobserved telemetry gaps and must be disclosed in the report.
3. **No Database Contamination:** Backend unit and integration tests must run against isolated test SQLite or throwaway PostgreSQL instances, never modifying the active monitoring platform database.
4. **Scope Control (Cut Priority):** If schedule limits arise, cut in this order:
   - Why-flagged feature explanation breakdown
   - Visualization plots in `results.md`
   - Traffic step variations in `M1`
   - Anomaly score gauge on UI
   - *Non-negotiable core:* The Isolation Forest detector, hybrid scheduler mode, and fallback mechanism must remain.